

# Shaili Seth

Roll 2405901 · GitHub `shailiseth24` · CSE-14 · Reviewer: Sinjan Mishra (Sprint A), Sohini Pandit (Sprint B)

You build the envelope every packet travels in, and the conversation wrapped around each transfer: the 20-byte header and its checksum, the handshake that opens a transfer and the FIN that closes it, the sender and receiver programs, and the simplest of the three protocols — Stop-and-Wait. Later you build the integrity suite that decides when Core is complete, and you run Experiment 1. **Your Thursday 17 deadline is the most important date in the sprint**, because no one can test a protocol until your packet format is merged.

### Thu 17 Sep

`Packet` rewritten and merged — blocks all four of us

### Sun 20 Sep

8 MiB Stop-and-Wait transfer matches through the emulator

### Thu 24 Sep

Your integrity suite decides Core complete

| YOU DEPEND ON                                                                | DEPEND ON YOU                                                                                   |
|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <b>Sohan</b> — emulator rewrite merged Thu 17, needed to test Stop-and-Wait  | <b>Everyone</b> — <code>Packet</code> on Thu 17                                                 |
| <b>Sinjan</b> — <code>RttEstimator</code> wired into Stop-and-Wait on Thu 24 | <b>Sinjan, Sohini</b> — <code>ArqProtocol</code> and <code>docs/CONVENTIONS.md</code> by Sat 19 |
|                                                                              | <b>Sohan</b> — the sender's <code>RESULT</code> line by Sun 20                                  |

## Contents

|                                      |                                    |
|--------------------------------------|------------------------------------|
| 1 · Get set up                       | 9 · Experiment 1                   |
| 2 · Your part of the system          | 10 · Your report section           |
| 3 · Concepts you must master         | 11 · How we work together          |
| 4 · Design decisions you must defend | 12 · Using AI without losing marks |
| 5 · Your day-by-day plan             | 13 · Viva preparation              |
| 6 · Team conventions                 | 14 · Help, references, glossary    |
| 7 · How to test your work            | 15 · Sprint checklist              |
| 8 · When something breaks            |                                    |

# 1 · Get set up — by Tuesday 15 September, 10 pm

Gate 1 is the first check of the sprint: all four of us have cloned the repository, verified our git identity, and pushed a first commit. Everything after it depends on this.

**The repository is not a shared folder.** Sohan's copy lives on Sohan's laptop. You make your own complete copy on *your* laptop by cloning it from GitHub, then you work, commit and push from there. GitHub is only the meeting point. Every command in this handbook runs on your own machine. (If you already have a working clone, start at step 4.)

## Step 1 — Install the tools

- **Git.** On Windows this also installs **Git Bash** — run every command in this handbook there, not in PowerShell or Command Prompt.
- **JDK 21** (Adoptium Temurin). Check that `java -version` and `javac -version` both report 21.
- **WSL2 with Ubuntu** on Windows, because the experiments run on Linux. Inside Ubuntu install the JDK and Python too:

```
wsl --install -d Ubuntu
wsl -d Ubuntu
sudo apt update && sudo apt install -y openjdk-21-jdk make python3-pip
java -version
```

## Step 2 — Clone onto your laptop

```
git clone https://github.com/sohan1611/rdt-udp.git
cd rdt-udp
```

You now have your own `rdt-udp` folder. Everything from here on happens inside it.

## Step 3 — Tell git who you are, inside that folder

```
git config user.name "Shaili Seth"
git config user.email "the-email-on-your-github-account"
```

**This email decides whether you count as a contributor.** It must be one of the addresses listed at [github.com/settings/emails](https://github.com/settings/emails) for your account `shailiseth24`. If you would rather not use a personal address, that page shows a private one ending in `@users.noreply.github.com` that links just as well. Commits with an unrecognised email show a grey avatar and are credited to nobody — and sir treats commit history as evidence of individual contribution.

## Step 4 — Confirm the build

```
./build.sh
```

You should see three test suites and **33 tests, 0 failed**. If anything fails, paste the output in the group chat the same evening.

## Step 5 — Push your first commit

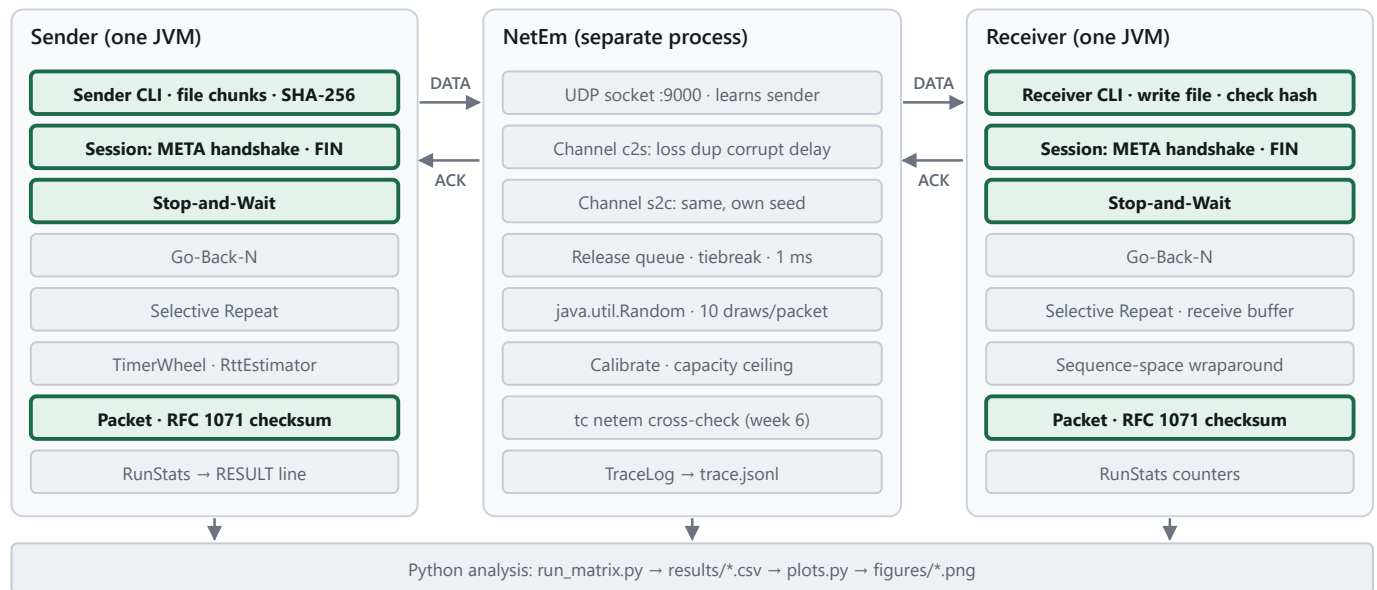
Open `AI-USE.md` and replace `__[name]_` beside your role heading with your name. Then:

```
git checkout -b setup/shaili
git add AI-USE.md
git commit -m "Add Shaili to AI-USE.md"
git push -u origin setup/shaili
git log -1 --format='%an <%ae>'
```

The last command must print your own name and your GitHub email. Open the pull request on GitHub and ask Sinjan Mishra to approve it. Your first push asks you to sign in to GitHub; on Windows a browser window opens, which is normal.

**Done when:** your PR is merged, `git log` shows your own name and email, and you can see yourself on the repository's Contributors page.

## 2 · Your part of the system



Bold, coloured boxes are yours. Grey boxes belong to teammates — either you depend on them, or they depend on you. Packets always travel through NetEm: the sender and receiver never talk directly.

### Files you own

| FILE                                                       | STATUS                                    | WHAT IT DOES                                                                           |
|------------------------------------------------------------|-------------------------------------------|----------------------------------------------------------------------------------------|
| src/main/java/rdt/Packet.java                              | AI draft — rewrite by hand, Wed 16–Thu 17 | Encodes and decodes the 20-byte header; computes and verifies the checksum             |
| src/main/java/rdt/CorruptPacketException.java              | AI draft — rewrite by hand                | Signals a datagram that cannot be trusted: runt, bad version, bad length, bad checksum |
| src/main/java/rdt/ArqProtocol.java and ProtocolConfig.java | New, Fri 18                               | The interface all three protocols implement, and their shared settings                 |
| src/main/java/rdt/Session.java (suggested name)            | New, Fri 18 – Tue 22                      | META handshake, FIN teardown and hash check, wrapped around any protocol               |
| src/main/java/rdt/StopAndWait.java                         | New, Sat 19                               | Alternating-bit ARQ                                                                    |
| src/main/java/app/Sender.java, Receiver.java               | New, Sun 20                               | Command-line programs using the flags in convention 5                                  |
| src/test/java/rdt/PacketTest.java                          | Exists, 13 tests — keep                   | Your specification for the rewrite                                                     |
| src/test/java/rdt/IntegrityTest.java                       | New, Wed 23 – Thu 24                      | 3 protocols × 5 channel profiles × 10 seeds, asserting a hash match                    |
| docs/CONVENTIONS.md                                        | New, Sat 19                               | The decisions from the Friday call                                                     |

Ask the owner before changing emulator/\* (Sohan), TimerWheel, RttEstimator or GoBackN (Sinjan), or SelectiveRepeat and its receive buffer (Sohini).

## 3 · Concepts you must master

### 3.1 · The header, byte by byte

| OFFSET | SIZE | FIELD      | RULES                                                                   |
|--------|------|------------|-------------------------------------------------------------------------|
| 0      | 1    | version    | Always 1. Reject anything else.                                         |
| 1      | 1    | type       | DATA 0, ACK 1, SACK 2, FIN 3, FINACK 4                                  |
| 2      | 1    | flags      | Bit 0x01 = META, set only on the handshake packet                       |
| 3      | 1    | reserved   | Write 0; ignore when reading                                            |
| 4      | 4    | seq        | Unsigned 32-bit sequence number                                         |
| 8      | 4    | ack        | Unsigned 32-bit acknowledgement number (convention 3)                   |
| 12     | 2    | payloadLen | Must equal the bytes that arrived minus 20                              |
| 14     | 2    | window     | Unsigned 16-bit; informational in Core                                  |
| 16     | 2    | checksum   | RFC 1071, over the header with this field zero, followed by the payload |
| 18     | 2    | sackCount  | 0 in Core; used by Sohini's SACK stretch goal in week 7                 |

Everything is **big-endian** — network byte order, most significant byte first. `ByteBuffer` is big-endian by default, so as long as you never call `order(ByteOrder.LITTLE_ENDIAN)` you get network byte order for free, even though your laptop's processor is little-endian internally.

### 3.2 · Signed bytes, unsigned fields

Java has no unsigned integers. This is the bug most likely to cost you a day, so understand it before writing a line.

- A Java `byte` runs from  $-128$  to  $127$ . The byte `0xF0` reads as  $-16$ . Before combining bytes into a larger number, mask each one: `b & 0xFF`.
- `buf.getInt()` returns a signed `int`. A sequence number of 3,000,000,000 comes back as  $-1,294,967,296$ . Widen it with `Integer.toUnsignedLong(buf.getInt())` and keep it in a `long`.
- Writing is the easy direction: `buf.putInt((int) seq)` keeps the low 32 bits, which is exactly the bit pattern you want on the wire.
- The 16-bit fields work the same way with `Short.toUnsignedInt(buf.getShort())`.

|                        |             |                                        |
|------------------------|-------------|----------------------------------------|
| seq on the wire        | FF FF FF FF |                                        |
| buf.getInt()           | -1          | wrong                                  |
| Integer.toUnsignedLong | 4294967295  | right (PacketTest checks exactly this) |

### 3.3 · The Internet checksum, RFC 1071

- Take the datagram — header with the checksum field set to zero, then the payload — as a sequence of 16-bit big-endian words. If the length is odd, the last byte is the *high* half of a final word.
- Add the words using one's complement arithmetic: whenever a sum spills past 16 bits, add the carry back into the bottom. This is the **end-around carry**.
- Take the bitwise complement of the result and keep 16 bits. Write it into offset 16.

Work these by hand before you code, then check each against your implementation:

|           |                |                                                                                         |                                             |
|-----------|----------------|-----------------------------------------------------------------------------------------|---------------------------------------------|
| Example 1 | 45 00 00 1C    | $0x4500 + 0x001C = 0x451C$                                                              | complement <code>0xBAE3</code>              |
| Example 2 | FF FF 00 02    | $0xFFFF + 0x0002 = 0x1\_0001$<br>fold: $0x0001 + 0x0001 = 0x0002$                       | complement <code>0xFFFFD</code>             |
| Example 3 | 01 02 03 (odd) | $0x0102 + 0x0300 = 0x0402$                                                              | complement <code>0xFBFD</code>              |
| Example 4 | bytes 0..63    | word $i = 514i + 1$ , $\text{sum} = 254,976 = 0x3E400$<br>fold: $0xE400 + 0x3 = 0xE403$ | complement <code>0x1BFC</code> (PacketTest) |

**Verifying on receipt.** Either zero the field, recompute, and compare with what arrived — our choice, because it lets the error message show both values — or add every word including the checksum, which gives 0xFFFF for an intact datagram. The guarantee is the same.

**What it always catches:** any single-bit error. Flipping one bit changes one word by  $2^j$ , and  $2^j$  is never a multiple of 0xFFFF, so the one's complement sum must change. Flipping one bit of Example 1 to `45 01 00 1C` gives 0xBAE2 instead of 0xBAE3.

**What it misses:** two 16-bit words swapped (`00 1C 45 00` still gives 0xBAE3); errors in two words that cancel; and a word changing between 0x0000 and 0xFFFF, which are both zero in one's complement. That is one reason real transports also rely on sequence numbers — a checksum alone does not notice reordering.

### 3.4 · Stop-and-Wait — rdt 3.0

Only one packet is ever unacknowledged, so the sequence number only needs to answer one question: *is this the packet I just had, or the next one?* One bit is enough. After the handshake, your data packets are numbered 1, 0, 1, 0 ...

| SIDE AND STATE                   | EVENT                                  | ACTION                                            |
|----------------------------------|----------------------------------------|---------------------------------------------------|
| Sender, waiting for ACK <i>b</i> | ACK <i>b</i> arrives intact            | Stop the timer; next packet uses bit 1− <i>b</i>  |
|                                  | Timeout                                | Resend the same packet; restart the timer         |
|                                  | Corrupt packet, or ACK for 1− <i>b</i> | Ignore it; the timer keeps running                |
| Receiver, expecting <i>b</i>     | DATA <i>b</i> intact                   | Deliver it, ACK <i>b</i> , now expect 1− <i>b</i> |
|                                  | DATA 1− <i>b</i> intact — a duplicate  | <b>Do not deliver.</b> ACK 1− <i>b</i> again      |
|                                  | Corrupt packet                         | Drop it silently; the sender's timer recovers     |

**Why the receiver must re-ACK duplicates.** The data arrives and is delivered, but its ACK is lost. The sender times out and resends. If the receiver discards the duplicate without acknowledging it, the sender resends forever.

**Premature timeout.** If an ACK is only delayed, the sender resends, then receives both the original ACK and a duplicate. This is why the sender ignores an ACK for the other bit rather than treating it as progress.

**The throughput ceiling.** One packet per round trip means goodput can never exceed  $\text{payload} \times 8 \div \text{RTT}$ . With 1400 bytes and a 40 ms RTT that is **280 kbps, however fast the link**. That ceiling is a line on Figure 1.

### 3.5 · The session: handshake, teardown, hash

**Handshake.** The sender's first packet is DATA with `seq 0` and the META flag. Recommended payload: UTF-8 text of three lines — filename, size in bytes, SHA-256 in hex. The receiver rejects a filename containing a line break or a size that does not parse, and replies ACK 0. The META packet can be lost like any other, so the sender retransmits it on timeout and the receiver re-ACKs duplicates.

**Teardown.** Once the last data packet is acknowledged, the sender sends FIN. The receiver replies FINACK and then **lingers** for about three timeouts, answering any repeated FIN — because its FINACK can be lost exactly like a data ACK. The sender retransmits FIN on timeout and gives up after a fixed number of tries. Giving up is safe at that point, since the file has already arrived, but it must be logged.

**Hash.** `MessageDigest.getInstance("SHA-256")`, updated chunk by chunk — by the sender as it reads the file, by the receiver as it writes chunks in order. Compare the two hex strings at the end and report `sha256_match`.

**Chunking.** 8 MiB is 8,388,608 bytes: 5,992 packets of 1400 bytes, the last holding 1,208. Create test files with `head -c 8388608 /dev/urandom > test8m.bin` in Git Bash or WSL.

## 4 · Design decisions you must defend

| DECISION                           | WHY — THE ANSWER TO GIVE AT THE VIVA                                                                                                                                                |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A fixed 20-byte header             | Constant-size parsing with no branches, every field at a known offset, and a reserved field for SACK so the format never changes mid-project.                                       |
| Packet-numbered, not byte-numbered | Go-Back-N and Selective Repeat are defined in packets in the textbook, so window arithmetic stays in whole packets. TCP numbers bytes because it streams; we transfer fixed chunks. |

| DECISION                                                    | WHY — THE ANSWER TO GIVE AT THE VIVA                                                                                                                |
|-------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Verify the checksum before trusting any field               | A corrupted <code>payloadLen</code> could otherwise drive a read past the end of the buffer or misparses the payload.                               |
| Zero-and-recompute verification                             | Same guarantee as summing to 0xFFFF, but the exception can report both values when they disagree.                                                   |
| Session layer outside the protocols                         | One handshake and one teardown for all three protocols: less code, and identical session overhead, so experiments compare only the ARQ logic.       |
| A META flag, not just <code>seq 0</code>                    | In alternating-bit Stop-and-Wait, 0 is also an ordinary data number. The flag makes a late duplicate handshake impossible to mistake for data.      |
| Checksum written as RFC 1071 describes it                   | We measured 0.6 $\mu$ s per packet in Java, so there is nothing to gain from optimising it and a lot to gain from being able to explain every line. |
| Stop-and-Wait times out with the socket, not the timer heap | It only ever has one timer, so a socket receive timeout is the simplest correct tool. From Thu 24 the value comes from Sinjan's estimator.          |

## 5 · Your day-by-day plan

### Sprint A · 14–20 September

#### Mon 14 – Tue 15

Setup and reading

1. Complete section 1. First commit merged by Tuesday 10 pm.
2. Read `docs/PLAN.md` section 4.1 and RFC 1071 section 1. Read Kurose & Ross section 3.4.1 on rdt 3.0.
3. Draw the header from memory until you get every offset right.
4. Work the four checksum examples in 3.3 by hand on paper.

**Done when:** your first commit is merged and your hand-worked checksums match 3.3.

#### Wed 16 – Thu 17

Rewrite `Packet.java` · BLOCKS EVERYONE

1. `git checkout -b m1/packet-rewrite`
2. Read the draft once to see its shape. Then delete the bodies of `encode`, `decode` and `checksum`, and write them from 3.1–3.3. If you get stuck, go back to the tables, not to the old code.
3. **Write `checksum` first**, then run `./build.sh`: the known-value and single-bit-flip tests will tell you whether it is right before anything else depends on it. Commit.
4. **Then `encode`**: allocate 20 + payload bytes, put every field in offset order, compute the checksum over the whole array while its field is still zero, write it at offset 16. Commit.
5. **Then `decode`**, in this order: reject anything shorter than 20 bytes; verify the checksum; read the fields with the unsigned widening from 3.2; check the version; check `payloadLen` against the bytes that arrived. Commit.
6. When `PacketTest` shows 13 of 13, rewrite `CorruptPacketException` and open the PR by Thursday afternoon. Sinjan Mishra reviews it with three *why* questions.
7. Tick the `Packet.java` box in `AI-USE.md`, sign it and date it.

**Done when:** merged into `main` by Thursday night with `PacketTest: 13 run, 0 failed`, and the `AI-USE.md` box ticked.

**Be able to explain:** questions 1, 2, 3 and 8 in section 13.

#### Fri 18

Handshake, interface, conventions call

1. Add the `META` flag constant and the handshake payload format from 3.5, with a test that formats a payload and parses it back, and one that rejects a bad size.
2. Draft `ArqProtocol` and `ProtocolConfig` as described in convention 1. Keep it small: two operations and the settings. Push it as a draft PR so Sinjan and Sohini can read it before the call.
3. **8 pm — the conventions call.** You chair it. Go through section 6 row by row and write down every decision.

**Done when:** the handshake tests pass and the interface draft has been discussed on the call.

#### Sat 19

Conventions, then Stop-and-Wait

1. First thing: commit `docs/CONVENTIONS.md` with the call's decisions, and merge the interface once Sinjan and Sohini approve. They start building against it on Sunday and Monday.
2. Write `StopAndWait` from the table in 3.4 — sender first, then receiver.

**Done when:** `CONVENTIONS.md` and the interface are merged, and Stop-and-Wait compiles against them.

## Sun 20

CLIs and the first real transfer

1. Write `Sender` and `Receiver` with the flags from convention 5. At the end of a transfer the sender prints the single `RESULT` line from convention 8.
2. Run a real transfer through the emulator at 0% loss, in three terminals:

```
java -cp build/classes emulator.NetEm --listen 9000 --to 127.0.0.1:9001 --seed 1 --both delay=20
java -cp build/classes app.Receiver --port 9001 --out received.bin --protocol stopwait
java -cp build/classes app.Sender --file test8m.bin --to 127.0.0.1:9000 --protocol stopwait
sha256sum test8m.bin received.bin
```

**Done when:** both hashes match. That is your part of Gate 3, Sunday 10 pm. Expect it to take about four minutes — see 3.4.

## Sprint B · 21–27 September

### Mon 21 – Tue 22

Harden Stop-and-Wait; FIN teardown

1. Run at `--both loss=0.10,delay=20` across seeds 1 to 10. Every failure is a bug in the table in 3.4 — find which row.
2. Add FIN, FINACK and the receiver linger from 3.5. Test it by dropping FINACKs: run the emulator with `--down loss=0.5` and confirm the transfer still ends cleanly.
3. No fixed `Thread.sleep` anywhere in shutdown.

**Done when:** 10 of 10 seeds match at 10% loss, each ending with an explicit FIN exchange.

### Wed 23 – Thu 24

The integrity suite · decides Core complete

1. Wednesday: build `IntegrityTest` (design in section 7) running Stop-and-Wait, and Go-Back-N as soon as Sinjan merges it that night.
2. Thursday: add Selective Repeat the moment Sohini merges it, run the full suite, and post the result in the chat.

**Done when:** 3 protocols × 5 profiles × 10 seeds all match — **Gate 5, Core complete**, Thursday 10 pm.

### Fri 25 – Sun 27

Experiment 1

1. Friday: with Sohan, start the Stop-and-Wait and Go-Back-N cells first — Stop-and-Wait is the slow one, so it runs overnight. Section 9 has the design and the runtime numbers.
2. Saturday: add the Selective Repeat cells.
3. Sunday: generate Figure 1 with `plots.py`.

**Done when:** Figure 1 is generated by script from committed CSVs with at least 3 seeds per cell.

## After the sprint

### 28 Sep – 26 Oct

Buffer, then weeks 6–8

1. **28 Sep – 5 Oct:** Experiment 1 at the full 10 seeds. Write the methodology section with Sohan.
2. **6–12 Oct:** teach your module at the cross-teaching session; learn Go-Back-N and Selective Repeat well enough to answer section 13 of their handbooks. Draft your report section.
3. **13–19 Oct:** review the full report draft; check every figure caption says what the reader should notice.
4. **20–26 Oct:** code freeze 22 October; viva drills.

## 6 · Team conventions — agreed on the Friday 18 call

Four people are writing code that has to plug together on Monday 21. Each item below is a decision that, left unspoken, becomes a day of integration bugs. **On Friday 18 September at 8 pm, all four of us join a 30-minute call**, confirm or change each default, and Shaili commits the result as `docs/CONVENTIONS.md` on Saturday morning. Until then, treat these as proposals — and if you disagree with one, bring the reason to the call rather than quietly doing something else.

| #  | TOPIC                    | PROPOSED DEFAULT                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | OWNER                  |
|----|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| 1  | Protocol interface       | One <code>ArqProtocol</code> interface with two operations: <i>send a file to a peer</i> and <i>receive a file on a port</i> . Both take a shared <code>ProtocolConfig</code> (window, RTO mode, sequence-space bits, payload size) and return <code>RunStats</code> . The session layer — metadata handshake, FIN teardown, hash check — sits outside the protocols and wraps whichever one is chosen, so Go-Back-N and Selective Repeat never reimplement it.                                                                                                                                                                    | Shaili                 |
| 2  | Sequence numbers         | The handshake is a DATA packet with <code>seq = 0</code> and flag bit <code>0x01</code> (META) set. File data starts at <code>seq = 1</code> . The sequence space is $k$ bits, set with <code>--seqbits</code> ; the default is 32 and only wraparound tests make it small. Every comparison between sequence numbers uses modular arithmetic.                                                                                                                                                                                                                                                                                     | Shaili                 |
| 3  | What an ACK number means | The <code>ack</code> field always holds a sequence number that <i>was received</i> . Stop-and-Wait and Selective Repeat acknowledge exactly that packet. Go-Back-N's <code>ack n</code> is cumulative: everything up to and including $n$ arrived in order — the textbook convention. Each protocol restates its rule in its class comment.                                                                                                                                                                                                                                                                                        | Shaili, Sinjan, Sohini |
| 4  | Payload size             | 1400 bytes per DATA packet; the last one is shorter. An 8 MiB file is therefore 5,992 data packets plus the handshake.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Shaili                 |
| 5  | Command-line flags       | <b>Sender:</b> <code>--file</code> <code>--to host:port</code> <code>--protocol stopwait gbn sr</code> <code>--window N</code> <code>--rto adaptive fixed:X</code> <code>--seqbits k</code> <code>--verbose</code> .<br><b>Receiver:</b> <code>--port</code> <code>--out</code> <code>--protocol</code> <code>--window N</code> <code>--seqbits k</code> <code>--verbose</code> . An unknown flag is an error, never silently ignored.                                                                                                                                                                                             | Shaili                 |
| 6  | Ports                    | For manual runs the emulator listens on 9000 and the receiver on 9001. The experiment harness picks fresh ports for every run.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Sohan                  |
| 7  | Fixed RTO                | <code>--rto fixed:1.5</code> means $RTO = 1.5 \times \text{base RTT}$ , where <b>base RTT = emulator delay up + delay down</b> . With <code>--both delay=20</code> that is 40 ms, so <code>fixed:1.5</code> is 60 ms.                                                                                                                                                                                                                                                                                                                                                                                                              | Sinjan                 |
| 8  | Result line              | At the end of a transfer the sender prints exactly one line beginning <code>RESULT</code> followed by a single JSON object of <code>RunStats</code> fields. Nothing else may print a line starting with <code>RESULT</code> . <code>run_matrix.py</code> reads only that line.                                                                                                                                                                                                                                                                                                                                                     | Sohan                  |
| 9  | What goodput means       | <b>Goodput</b> = $\text{file bytes} \times 8 \div \text{elapsed seconds}$ , where <i>elapsed</i> runs from sending <code>seq 1</code> to receiving the ACK that covers the last data packet — handshake and FIN excluded.<br><b>Wire throughput</b> = every byte the sender put on the wire, headers and retransmissions included, $\times 8 \div$ the same elapsed time.                                                                                                                                                                                                                                                          | Sohan                  |
| 10 | RunStats fields          | <code>protocol</code> , <code>window</code> , <code>seqbits</code> , <code>rto_mode</code> , <code>file_bytes</code> , <code>elapsed_ms</code> , <code>goodput_bps</code> , <code>wire_bytes</code> , <code>throughput_bps</code> , <code>data_sent</code> , <code>retransmissions</code> , <code>timeouts</code> , <code>dup_acks</code> , <code>acks_received</code> , <code>corrupt_dropped</code> , <code>sha256_match</code> . The harness adds the run settings itself — <code>seed</code> , <code>loss</code> , <code>dup</code> , <code>corrupt</code> , <code>reorder</code> , <code>delay</code> , <code>jitter</code> . | Sohan                  |
| 11 | Trace lines              | <code>us</code> , <code>dir</code> , <code>type</code> , <code>seq</code> , <code>ack</code> , <code>action</code> , <code>delayMs</code> . The <code>type</code> and <code>ack</code> fields are new in the emulator rewrite. Today every ACK is logged with <code>seq 0</code> , so ACKs cannot be told apart — and Experiment 4 needs to tell them apart.                                                                                                                                                                                                                                                                       | Sohan                  |
| 12 | Logging                  | No printing per packet unless <code>--verbose</code> is given. Writing a line for every packet costs more than the protocol itself and makes goodput numbers meaningless.                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Everyone               |
| 13 | Tests and main           | Every new test class is added to <b>both</b> <code>build.sh</code> and the <code>TESTS</code> line of the <code>Makefile</code> , or it never runs. Run <code>./build.sh</code> before every push: <code>main</code> must always build, because a compile error there stops all four of us.                                                                                                                                                                                                                                                                                                                                        | Everyone               |

| #  | TOPIC               | PROPOSED DEFAULT                                                                                                                                                                                                                                                                                     | OWNER  |
|----|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| 14 | Sequence arithmetic | One small <code>SeqSpace</code> helper for every modular comparison — distance from the window base, "inside the window", "inside the previous window" — built on <code>Math.floorMod</code> and shared by Go-Back-N and Selective Repeat. The wraparound rule is then written once and tested once. | Sohini |

## 7 · How to test your work

### Tests you already have — PacketTest, 13 tests

Round trips with and without payload, an odd-length payload, sequence numbers above  $2^{31}$ , maximum 16-bit values, **every single-bit flip of a 148-byte packet**, runt packets, a mismatched length, an unknown version, the RFC 1071 end-around property, a known checksum value, and oversized or out-of-range fields. Treat them as the specification: a rewrite is correct when they all pass.

### Tests to write

| TEST CLASS                   | WHAT IT MUST PROVE                                                                                                                                            |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>HandshakeTest</code>   | Format then parse gives the same name, size and hash; a bad size is rejected; a name containing a line break is rejected.                                     |
| <code>StopAndWaitTest</code> | Transfers a small file correctly over a clean channel, 10% loss, 5% duplication, 5% corruption and 10% reordering. Zero retransmissions on the clean channel. |
| <code>SessionTest</code>     | The transfer ends cleanly when half the FINACKs are dropped; the receiver answers a repeated FIN during its linger.                                           |
| <code>IntegrityTest</code>   | For every protocol × profile × seed, the received hash matches.                                                                                               |

**Run the emulator inside the test.** `NetEmSmokeTest` already shows how: start `NetEm` on a daemon thread with port 0, call `awaitReady`, and read `boundPort()`. Every test then gets free ports and never collides with another run. Read that fixture before writing yours.

**Keep unit tests fast.** Use a 256 KiB file in tests, not 8 MiB — the integrity suite runs 150 transfers. The 8 MiB file is for the experiments.

### Profiles for the integrity suite

| PROFILE     | EMULATOR SETTING                             |
|-------------|----------------------------------------------|
| clean       | <code>delay=5</code>                         |
| lossy       | <code>loss=0.10, delay=5</code>              |
| duplicating | <code>dup=0.05, delay=5</code>               |
| corrupting  | <code>corrupt=0.05, delay=5</code>           |
| reordering  | <code>reorder=0.10, delay=5, jitter=2</code> |

Remember convention 13: add every new test class to `build.sh` and to the `Makefile`.

## 8 · When something breaks

| SYMPTOM                                     | LIKELY CAUSE                                                                                                                 | HOW TO CHECK                                                      |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| Every packet fails its checksum             | Checksum computed after the field was filled, written at the wrong offset, or bytes combined without <code>&amp; 0xFF</code> | <code>Packet.checksum</code> over an encoded packet must return 0 |
| Sequence numbers negative or enormous       | Missing <code>Integer.toUnsignedLong</code> , or a cast in the wrong place                                                   | <code>PacketTest</code> 's above- $2^{31}$ test                   |
| Good packets rejected as "declared payload" | Decoding the whole 65,535-byte receive buffer instead of <code>DatagramPacket.getLength()</code> bytes                       | Print the length you pass to <code>decode</code>                  |

| SYMPTOM                                               | LIKELY CAUSE                                                                                      | HOW TO CHECK                                                                        |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Transfer hangs before any data                        | Handshake lost and not retransmitted; or the sender aimed at 9001 instead of the emulator on 9000 | Run the emulator with <code>--verbose</code>                                        |
| Emulator logs traffic, receiver gets nothing          | <code>--to</code> points at the wrong port, or the receiver was started second                    | Start the receiver first                                                            |
| Received file larger than the original                | The receiver delivered a duplicate instead of re-ACKing it                                        | Rows 5 and 6 of the table in 3.4                                                    |
| Hangs at the very end                                 | FINACK lost and the sender never retransmits FIN; or the receiver exited without lingering        | SessionTest with dropped FINACKs                                                    |
| Same size, different hash                             | Delivered out of order, or accepted a corrupt packet before checking                              | Verify the checksum before any delivery                                             |
| Goodput far below 280 kbps at 0% loss                 | Printing every packet, or a timeout of a second or more                                           | Run without <code>--verbose</code> ; print the RTO                                  |
| Address already in use                                | A previous run still holds the port                                                               | Git Bash: <code>taskkill //F //IM java.exe</code> ;<br>WSL: <code>pkill java</code> |
| <code>bash\r: No such file or directory</code> in WSL | <code>build.sh</code> checked out with Windows line endings                                       | Re-clone inside WSL; <code>.gitattributes</code> now forces Unix line endings       |

## 9 · Experiment 1 — goodput against loss rate

|            |                                                                                                                                                          |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Question   | How does each protocol's goodput fall as packet loss rises?                                                                                              |
| Variable   | Loss 0, 0.1, 0.5, 1, 2, 5, 10, 20%, applied in both directions with <code>--both loss=p</code>                                                           |
| Held fixed | Window 32 for Go-Back-N and Selective Repeat; 1400-byte payload; 20 ms delay each way; jitter, RTO mode and file size as locked in the configs on Tue 22 |
| Seeds      | At least 3 per cell by Sun 27; 10 in the buffer week                                                                                                     |
| Measured   | <code>goodput_bps</code> , <code>throughput_bps</code> , <code>retransmissions</code> , <code>timeouts</code>                                            |
| Owner      | You, with Sohan running the harness                                                                                                                      |

### What we expect, and what would be interesting

- **Stop-and-Wait** stays under its 280 kbps ceiling at every loss rate, falling as loss rises.
- **Go-Back-N** is fastest at 0% but falls steeply, because each loss can cost up to a whole window of resends.
- **Selective Repeat** falls gently, because each loss costs one resend.
- The claim models normalised goodput as roughly  $(1-p)$  for Selective Repeat and  $(1-p)^W$  for Go-Back-N. That model is crude — it ignores ACK loss and the time spent waiting for timeouts — so where the measured curve departs from it, **explain why**. That analysis is worth more than a curve that happens to fit.

### Runtime — plan for it

**Stop-and-Wait is the bottleneck of the whole experiment.** Estimated at 8 MiB, RTO around 120 ms, loss in both directions: **4 minutes a run at 0% loss, about 11 minutes at 20%**. One seed of the eight Stop-and-Wait cells takes about 44 minutes; ten seeds, about 7.3 hours. At 2 MiB those become 1 and 2.7 minutes, and 1.8 hours for ten seeds. So the Stop-and-Wait cells start **Friday evening** and run overnight. If Sohan's dry run on Tuesday 22 shows they are too slow, the whole experiment — every protocol, not just Stop-and-Wait — moves to 2 MiB, and the report says so. Goodput is a rate, so file size changes the running time, not the number being measured.

### Figure 1

- Left panel: goodput in Mbps against loss rate, one line per protocol, 95% confidence-interval error bars, and the 280 kbps Stop-and-Wait ceiling drawn as a dashed line.
- Right panel: goodput normalised to its own 0% value, with the  $(1-p)$  and  $(1-p)^{32}$  model curves overlaid.
- The loss axis includes 0, so it cannot be logarithmic — use evenly spaced categories.
- The caption states the settings and  $n$ , the number of seeds per point.

## 10 · Your report section — wire format and session

1. **Header layout** — the table from 3.1 and the reasons from section 4.
2. **The checksum** — the algorithm, one worked example, what it catches and what it cannot, and why we verify first.
3. **Unsigned fields in Java** — the problem and the fix, in a short paragraph with the `FF FF FF FF` example.
4. **The session** — handshake, teardown and hash, with a timeline figure of a lost FINACK.
5. **Stop-and-Wait** — the sender and receiver rules, and the throughput ceiling.
6. **Experiment 1** — Figure 1 and three paragraphs: what happened, how it compares with the model, and why.

About two to three pages. Draft it in week 6 while the cross-teaching is fresh.

## 11 · How we work together

### Nightly check-in — by 10 pm, in the group chat

Three lines, every night of the sprint. For example:

```
Done:      receive buffer releases 1,3,4,2 in order; 5 tests passing
Tomorrow:  duplicate handling, then open the PR
Blocked:   nothing
```

In a two-week sprint, a silent day costs whoever is waiting on you half a day. If you are blocked, say so that evening — never save it for the next gate.

### Branches

Never commit directly to `main`. Use one branch per task, named after your role — `m1/short-task-name`. Always start from the latest `main`:

```
git checkout main
git pull
git checkout -b m1/short-task-name
```

Bring `main` into your branch at least once a day, so you meet other people's changes while the conflicts are still small:

```
git fetch origin
git merge origin/main
```

### Pull requests

Push your branch, open a pull request on GitHub, and use this description:

```
What:      one sentence
Why:      which playbook task this is
How tested: the commands you ran and what they printed
Ask me:    two things you want the reviewer to question
```

| WHEN                       | YOUR REVIEWER — AND WHOSE PRS YOU REVIEW |
|----------------------------|------------------------------------------|
| Sprint A · 14–20 September | Sinjan Mishra                            |
| Sprint B · 21–27 September | Sohini Pandit                            |

**Review within 12 hours.** An unreviewed PR blocks someone. If your partner is stuck, anyone else may review instead, and you review your partner's work on the same terms.

**The explanation check.** Before approving, the reviewer asks two or three *why* questions — *why this data structure, what happens if this ACK is lost, why this bound and not another*. If the author cannot answer, the PR does not merge. A five-minute voice call is enough. It rehearses the viva every day, and it catches copy-pasted code while there is still time to rewrite it.

### Commits

- At least **one commit every working day** of the sprint, under your own name and email.
- Small commits whose messages say what changed — *"Release buffered packets once the gap at rcv\_base fills"*, not *"update"*.
- Never commit on someone else's behalf. The brief counts that as non-contribution by the absent member.
- No Co-Authored-By lines, no bot pull requests, and Dependabot stays off.** The contributor list must show exactly our four names.

### Other people's files

If you need a change in a file someone else owns, message them first. If it is urgent and they are unreachable, make the smallest change you can, say so in your PR, and tag them. A merge conflict in a file is resolved by that file's owner.

## 12 · Using AI without losing marks

### PERMITTED BY THE BRIEF

Explaining concepts · debugging · reviewing your code · generating tests · boilerplate and plotting scripts · improving your writing

### NOT PERMITTED

Generating the core protocol implementation and submitting it as your own work

**Our rule: use AI to understand your module, then type the code yourself.** Two reasons, both about marks. The viva is individual and worth 5 of the 10 marks, and sir will ask you to change your code live — code you did not write is code you cannot change under pressure, and it scores zero however well it runs. And a chatbot asked for a textbook protocol gives every group roughly the same code. Another group in our year is doing P3, and the brief treats identical-looking submissions as collusion.

### Uses that help

- *"Explain Karn's algorithm with a timeline where the ACK arrives just after a retransmission."* Then close it and draw the timeline yourself.
- *"Here is my window check and a failing test. Why does it fail when the sequence numbers wrap?"* — debugging code you wrote.
- *"Write tests for a receive buffer that must release packets in order."* Tests are explicitly permitted.
- *"Quiz me on my module the way a viva examiner would."* The single most useful thing it can do for you.

### Uses that cost marks

- *"Write GoBackN.java."* Even if you read it afterwards, you will not be able to change it live.
- Putting AI output into a PR you cannot then explain in the review check.

### Log it the same day

Whenever AI materially helped, add an entry under your role in `AI-USE.md`:

```
### 2026-09-17 – receive buffer
**What:** asked why Java's % operator goes negative for a wrapped sequence number
**Permitted under:** explaining concepts
**Mine:** the buffer design, all of the code, and the tests for the wrap case
```

Honest disclosure carries no penalty. Undisclosed use discovered at the viva does.

## 13 · Viva preparation

Practise these out loud with a teammate. Sir will also ask you to change code live — the last two are rehearsals.

### 1. Walk me through the checksum, byte by byte.

- 16-bit big-endian words; an odd final byte is the high half of a word.
- One's complement addition with the end-around carry; complement at the end.
- Computed with the checksum field at zero. Work Example 1 or 2 from 3.3 on the board.

### 2. Why is every single-bit error caught? Which errors are not?

- One flipped bit changes one word by  $2^j$ , which is never a multiple of  $0xFFFF$ , so the sum changes.
- Missed: swapped words, errors that cancel, a word flipping between  $0x0000$  and  $0xFFFF$ .

### 3. Why `Integer.toUnsignedLong` on the sequence field?

- Java's `int` is signed; above  $2^{31}$  the value reads as negative.
- Show `FF FF FF FF` reading as  $-1$  without it and 4,294,967,295 with it.

### 4. In Stop-and-Wait, an ACK is lost. What happens, step by step?

- The data was already delivered. The sender's timer fires and it resends.
- The receiver sees a duplicate, does not deliver it, and ACKs it again. The sender moves on.

### 5. Why is one bit enough for Stop-and-Wait but not for Go-Back-N?

- Stop-and-Wait has one packet in flight, so it only distinguishes "this one again" from "the next one".
- Go-Back-N has up to  $W$  in flight, and needs at least  $W+1$  distinct numbers to tell them apart.

### 6. Why can't Stop-and-Wait go faster than 280 kbps here?

- One packet per round trip:  $1400 \times 8 \div 0.040 \text{ s}$ . The link speed never enters the formula.

### 7. The FINACK is lost. What happens?

- The sender's FIN timer fires and it resends FIN; the lingering receiver answers again.
- If every FINACK is lost the sender gives up after its retry limit — safe, because the file has already arrived.

### 8. Why verify the checksum before reading the length field?

- A corrupted length could send the parser past the end of the buffer or mis-split header and payload. Nothing is trusted until the checksum passes.

### 9. Live change: add a NAK packet type.

- A new type constant and name; handling in the protocol that sends and receives it; a test.
- The header layout and the checksum do not change at all — explain why.

### 10. Live change: make the payload 1000 bytes instead of 1400.

- It is a `ProtocolConfig` value, so no header change. More packets per file, and Stop-and-Wait's ceiling drops to 200 kbps.

## 14 · Help, references and glossary

### Who to ask

| QUESTIONS ABOUT                                                                   | ASK                           | GITHUB                       |
|-----------------------------------------------------------------------------------|-------------------------------|------------------------------|
| Packet format, checksum, handshake, FIN, CLIs, Stop-and-Wait, the integrity suite | <a href="#">Shaili Seth</a>   | <a href="#">shailiseth24</a> |
| Timers, RTT estimation, Karn's algorithm, Go-Back-N, fixed and adaptive RTO       | <a href="#">Sinjan Mishra</a> | <a href="#">Thesin-01</a>    |
| Selective Repeat, the receive buffer, window bounds, wraparound                   | <a href="#">Sohini Pandit</a> | <a href="#">sohinix</a>      |
| The emulator, trace files, calibration, RunStats, CSVs, plots, repository access  | <a href="#">Sohan Mandal</a>  | <a href="#">sohan1611</a>    |

Stuck for more than an hour? Say so in the group chat. An hour stuck in a two-week sprint is expensive; two minutes of someone else's attention usually is not.

### References

- **RFC 1071**, *Computing the Internet Checksum* — section 1 is the algorithm.
- **RFC 6298**, *Computing TCP's Retransmission Timer* — Jacobson/Karels estimation, Karn's algorithm, backoff.
- **Kurose & Ross**, *Computer Networking: A Top-Down Approach* — section 3.4 (reliable data transfer: rdt 3.0, Go-Back-N, Selective Repeat) and section 3.5 (TCP round-trip time estimation).
- **Java 21 API documentation** — [java.nio.ByteBuffer](#), [java.net.DatagramSocket](#), [java.util.PriorityQueue](#), [java.security.MessageDigest](#), [java.util.Random](#), [Math.floorMod](#).
- **In the repository** — [docs/PLAN.md](#) for the design decisions, [docs/member-playbook.html](#) for everyone's dates, [AI-USE.md](#), and the existing tests, which show the house test style.

### Glossary

|                         |                                                                                                                                                  |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| ARQ                     | Automatic Repeat reQuest: reliability by detecting loss and retransmitting. Stop-and-Wait, Go-Back-N and Selective Repeat are three ARQ schemes. |
| RTT                     | Round-trip time — data out, ACK back.                                                                                                            |
| RTO                     | Retransmission timeout — how long the sender waits for an ACK before resending.                                                                  |
| SRTT, RTTVAR            | The smoothed RTT and its smoothed variation, from which RTO is computed (RFC 6298).                                                              |
| Cumulative ACK          | "Everything up to $n$ has arrived." Used by Go-Back-N.                                                                                           |
| Selective ACK           | "This particular packet has arrived." Used by Selective Repeat.                                                                                  |
| Window                  | How many unacknowledged packets the sender may have in flight at once.                                                                           |
| Sequence space          | The range of sequence numbers: $2^k$ values for $k$ bits, wrapping back to 0.                                                                    |
| Goodput                 | Useful file bytes delivered per second — excluding headers and retransmissions.                                                                  |
| Throughput              | Every byte put on the wire per second — including headers and retransmissions.                                                                   |
| Spurious retransmission | Resending a packet whose original actually arrived.                                                                                              |
| Network byte order      | Big-endian: most significant byte first. <a href="#">ByteBuffer</a> 's default.                                                                  |
| One's complement sum    | Addition where a carry out of the top bit is added back in at the bottom — the end-around carry. The basis of the Internet checksum.             |
| Seed                    | The starting value for the emulator's random generator. Same seed, same decisions, on any machine.                                               |
| 95% confidence interval | A range around a measured mean reflecting run-to-run variation. Every data point we plot carries one.                                            |
| JIT warm-up             | The JVM runs code slowly at first and compiles hot paths as it goes, so the first packets of a run can be slower.                                |

## 15 · Sprint checklist

- ☐ **Tue 15** — cloned, identity set, `./build.sh` green, first commit merged
- ☐ **Tue 15** — the four checksum examples worked by hand
- ☐ **Thu 17** — `Packet` rewritten, 13/13, merged; `AI-USE.md` box ticked
- ☐ **Fri 18** — handshake format and tests; interface draft PR; chaired the 8 pm call
- ☐ **Sat 19** — `CONVENTIONS.md` and `ArqProtocol` merged
- ☐ **Sun 20** — Sender and Receiver CLIs; `RESULT` line; 8 MiB transfer matches at 0% loss
- ☐ **Tue 22** — 10/10 seeds match at 10% loss; FIN teardown survives dropped FINACKs
- ☐ **Wed 23** — integrity suite running Stop-and-Wait and Go-Back-N
- ☐ **Thu 24** — integrity suite passes all three protocols: Core complete
- ☐ **Sun 27** — Figure 1 from at least 3 seeds per cell
- ☐ **Every night** — three-line check-in by 10 pm; at least one commit every working day